Coursework 1: Game Concept and Design

Temporal Paradox

Student name: [Your name]
Student ID: [Your student ID]
Module: Game Programming
Submission format: PDF

---

1. Game Title

Temporal Paradox

2. One-Sentence Game Idea

Temporal Paradox is a 2D puzzle-platform game in which the player records short versions of their own movement and replays them as time clones in order to solve pressure-plate, door, platform, crate, and hazard puzzles.

3. Intended Player Experience

The intended experience is a compact puzzle-platformer built around planning, timing, and spatial reasoning. The player should initially read each level as a simple platform challenge, but then realise that the level cannot be completed by the live character alone. Progress depends on thinking about the player character as both a current actor and a future helper. This gives the game a light "cause and effect" structure: the player performs an action, records it, and later uses that recorded action as part of the solution.

The tone should be clear and slightly mysterious rather than fast or chaotic. The game should encourage experimentation, because failed attempts are not treated as major punishment. A quick reset allows the player to try another recording route, improve the timing, or use a different object such as a crate. The main feeling I want to create is the satisfaction of solving a problem by coordinating with a past version of oneself.

4. Core Mechanic

The core mechanic is **recording and replaying the player's movement as a time clone**.

The player can press a key to start recording a short timeline of movement, jumping, and position. When the recording stops, the timeline is stored. The player can then spawn a clone that replays the stored movement. This clone exists physically in the level and can activate pressure plates, help

open doors, and support multi-step puzzle solutions. The mechanic is limited by a maximum number of active clones, so the player must decide which actions are worth recording.

5. What the Player Does Moment to Moment

Moment to moment, the player moves left and right, jumps between platforms, observes the state of doors and pressure plates, and records short action sequences. A typical loop is:

1. Read the level layout and identify blocked paths or inactive platforms.
2. Move the live character to a useful position, such as a pressure plate.
3. Start recording, perform the required movement, then stop recording.
4. Spawn the recorded clone and observe whether it completes the intended task.
5. Continue through the level while the clone repeats the recorded action.
6. Reset and adjust the plan if the timing or position is incorrect.

This loop keeps the controls simple while making the decision-making more interesting. The player is not only reacting to obstacles; they are designing a small sequence of actions that will later run without their direct control.

6. Target Player

The target player is someone who enjoys short puzzle-platform games and systems that can be understood through experimentation. The game is suitable for players who are comfortable with basic 2D movement but do not require advanced platforming skills. The main challenge is logical rather than reflex-based.

The likely audience includes:

- Students or casual PC players looking for a small but thoughtful game.
- Players who enjoy puzzle games such as Portal, Braid, or The Swapper.
- Players who like mechanical clarity and levels that can be solved through trial and error.

The prototype is designed for keyboard control on PC, using Unity's 2D physics and simple on-screen prompts.

7. Reference Games or Inspirations

The design is influenced by several existing games, but it uses them as design references rather than direct copies.

Braid is a useful reference because it shows how time manipulation can become the central puzzle language of a platform game. Temporal Paradox does not use full rewind, but it shares the idea that time can be treated as a playable resource.

The Swapper is relevant because it uses player copies to solve spatial puzzles. The inspiration here is the idea of separating the player's current body from another body that can still influence the level.

Portal is a reference for level readability and puzzle escalation. Although the mechanic is different, Portal demonstrates how a simple rule can be introduced, tested, and then combined with other elements across later levels.

Celeste is a secondary reference for responsive 2D movement. Temporal Paradox should not be as mechanically demanding, but the movement should still feel immediate and reliable because puzzle solving becomes frustrating if basic movement feels imprecise.

8. What Is Original or Creative About the Idea

The creative element is the way the game combines platforming with recorded self-cooperation. The clone is not just an extra character controlled at the same time as the player. Instead, it is a fixed performance created by the player's earlier decision. This changes the role of the player from direct controller to designer of a short timeline.

The originality of the prototype also comes from its small-scale puzzle grammar. Pressure plates, doors, switchable platforms, crates, and hazards are familiar elements, but they become more interesting when they must be coordinated with delayed actions. A plate may need a clone, a crate may need to hold a different plate, and a dangerous route may require a clone to perform a task before disappearing. The puzzle is therefore about arranging actions across time, not only moving through space.

9. Vertical Slice Plan

The vertical slice should show the complete intended experience in a small amount of content. It does not need a full campaign, but it must prove that the time-clone mechanic works and can support more than one type of puzzle.

The planned vertical slice contains:

- A main menu and short story introduction.
- Three playable 2D levels.
- A live player character with horizontal movement and jumping.
- A recording system that stores position, velocity, movement input, and jump input.
- A clone system that replays stored timelines.
- Pressure plates, doors, crates, switchable platforms, goals, and hazard zones.
- Simple visual feedback, including animated fire traps and clear door/plate states.
- A short ending scene after the final level.

The first level should teach the basic recording and plate-door relationship. The second level should add hazards, vertical movement, and a switchable bridge. The third level should combine crates and clone coordination, requiring the player to think about multiple objects and routes together.

10. Must-Have, Should-Have, Could-Have, and Cut-First Features

Must-Have Features

- 2D player movement with jumping and collision.
- A record and replay system for player timelines.
- Spawnable replay clones with a clear limit.
- Pressure plates that respond to the player, clones, and crates.
- Doors that open when the required plates are pressed.
- Reset functionality so failed puzzle attempts can be retried quickly.
- At least one complete level with a clear goal and win condition.

Should-Have Features

- Three levels with increasing puzzle complexity.
- Switchable platforms linked to pressure plates.
- Pushable crates as an alternative way to hold plates.
- Hazards that reset the player or stop clones.
- Basic story framing through intro and ending scenes.
- On-screen instructions showing recording status and available controls.

Could-Have Features

- A more polished timeline user interface.
- Sound effects for recording, clone spawning, doors, plates, and hazards.
- Better animation states for idle, walking, jumping, and clone playback.
- Level-select menu after completing the game.
- Ghost trail or transparency effects to make clones more readable.

Cut-First Features

- Complex combat or enemies.
- Online features, leaderboards, or accounts.
- Procedural level generation.
- Large narrative cutscenes.
- Advanced visual effects that do not directly support puzzle readability.

These cuts keep the project focused on the main mechanic. If development time becomes limited, the priority should be a stable and understandable clone-puzzle loop rather than adding unrelated content.

11. Unity Development Plan

The project will be developed in Unity as a 2D game using Rigidbody2D, BoxCollider2D, SpriteRenderer, prefabs, and scene-based level progression. Development should begin with the core player controller and then add the recording system, because every later puzzle system depends on the player and clone behaviour being stable.

The development order is:

1. Build the player controller, including horizontal movement, jumping, collision, and reset behaviour.
2. Implement the replay recorder, storing a short list of sampled frames.
3. Implement the replay clone, reading stored frames and applying position, velocity, and input.
4. Add the clone manager, including clone spawning, clone limits, and cleanup.
5. Build basic puzzle objects: pressure plates, doors, and goals.
6. Add secondary puzzle objects: crates, hazards, and switchable platforms.
7. Build level scenes and test puzzle readability.
8. Add UI feedback for controls, clone count, recording status, and level completion.
9. Add menu, intro, and ending scenes.
10. Polish visuals, fix bugs, and test the full playthrough.

The project should use prefabs and reusable scripts rather than making each level entirely separate. This will reduce repeated work and make later adjustments easier.

12. Main Systems and Scripts Expected to Build

The main systems are:

PlayerController2D: Handles live player movement, jumping, replay mode, forced replay state, and reset behaviour.

ReplayRecorder2D: Records the live player's timeline by sampling position, velocity, movement input, and jump input at short intervals.

RecordedFrame and RecordedTimeline: Store the data used by clone replay. These are the data structures behind the time-clone mechanic.

ReplayClone2D: Plays back a recorded timeline and applies the stored frames to a clone character.

CloneManager: Spawns clones, configures their visuals and colliders, tracks active clones, and removes them when the level resets.

TemporalParadoxGame: Controls the main level flow, including recording input, clone spawning, reset, level clear state, and scene transition.

PressurePlate2D: Detects whether the player, a clone, or a crate is pressing a plate.

Door2D and DualPlateDoor2D: Manage door state and link doors to pressure-plate conditions.

PushableCrate2D: Provides physics-based puzzle objects that can hold plates.

SwitchablePlatform2D: Enables or disables platforms based on plate state.

HazardZone2D: Resets the player or stops clones when they touch danger areas.

Goal2D: Detects level completion when the player reaches the goal with the door condition satisfied.

VisualFeedback, TimelineVisualizer, SpriteFrameAnimator2D, UIImageFrameAnimator, MainMenuController, and StoryIntroController: Provide supporting UI, animation, scene, and feedback systems.

13. Asset and Resource Plan

The asset plan is deliberately small so the project remains achievable. Most assets will be 2D sprites stored in Unity's Resources folder. The current plan includes character sprites, clone sprites, door sprites, ground and platform tiles, cloud backgrounds, lava or fire-trap animation frames, and simple UI elements.

Key resource categories:

- **Characters**: Player and clone sprites, using different visual roles so the player can distinguish live and replay characters.
- **Environment**: Grass ground, stone platforms, layered sky and cloud background.
- **Puzzle objects**: Doors, pressure plates, crates, switchable platforms, and goals.
- **Hazards**: Lava and animated fire trap frames.
- **UI**: Main menu, recording feedback, clone count, simple instructional text, and story screens.

The visual style should prioritise readability. Puzzle objects must be recognisable at a glance: plates should visibly change when pressed, doors should clearly fade or open, and hazards should be visually distinct from safe platforms. If extra time is available, polish should improve clarity before decoration.

14. Legal, Ethical, Social, Accessibility, and Security Considerations

Legal considerations mainly concern asset use. Any third-party sprites, fonts, audio, or visual effects must be either original, public domain, licensed for student use, or properly credited. The project should avoid copyrighted characters, music, or recognisable commercial assets unless permission is clear. If AI-generated or downloaded assets are used, their licence and allowed use should be checked before submission.

Ethically, the game does not include gambling, personal data collection, online chat, or manipulative monetisation. The design is single-player and offline, which reduces privacy and security concerns. Since the game uses no accounts or network services, it should not collect or store personal information.

Accessibility should be considered even in a small prototype. The controls should be simple and visible in the interface. Failure should be recoverable through a quick reset. Puzzle states should be communicated visually through position, colour, and object state rather than colour alone where possible. The game should avoid excessive flashing effects, and hazards should be readable before they punish the player.

Socially, the game has a neutral theme and does not rely on stereotypes or sensitive real-world topics. The main theme of cooperating with past versions of oneself can be presented as a puzzle idea rather than a serious psychological claim.

15. Development Schedule or Milestone Plan

Milestone 1: Core Prototype

Create the player controller, basic level scene, camera, ground collision, and reset behaviour. At the end of this milestone the player should be able to move, jump, collide with platforms, and restart from the starting position.

Milestone 2: Recording and Clone Replay

Implement the recorder, recorded timeline data, clone playback, and clone spawning. Test that clones follow the recorded route closely enough to activate level objects reliably.

Milestone 3: Basic Puzzle Loop

Add pressure plates, doors, and goals. Build a first level where the player must record a clone standing on a plate while the live player reaches the exit.

Milestone 4: Expanded Puzzle Objects

Add crates, hazards, and switchable platforms. Build at least two additional levels that combine these systems with the clone mechanic.

Milestone 5: Interface and Scene Flow

Add main menu, story intro, ending scene, on-screen controls, recording status, clone count, and level transitions.

Milestone 6: Testing and Polish

Playtest each level from a fresh start. Fix collision problems, unclear instructions, clone spawning issues, and unfair hazard placement. Improve sprites, animation feedback, and level readability where time allows.

Conclusion

Temporal Paradox is designed as a focused Unity 2D puzzle-platformer where the central idea is not only moving through a level, but preparing actions that a future clone will repeat. The design is achievable because it uses a small set of reusable systems, but it still offers meaningful puzzle depth through the combination of recording, replay, physical objects, and timed level states. The most important success criteria are that the clone mechanic feels reliable, the levels clearly teach their rules, and the player can solve each puzzle through observation and experimentation.